coursera | VANDERBILT UNIVERSITY

AI Agents and Agentic AI with Python & Generative AI

Module 1
Agentic AI Concepts

- Introduction
 Video • 10 min
- Flipped Interaction Pattern
 Video • 7 min
- The Agent Loop
 Video • 8 min
- Knowledge Check: Agent Loop & Flipped Interaction
 Practice Assignment • Grade: 100%
- **Running the Code Samples in the Course**
 Reading • 10 min
- Programmatic Prompting for Agents
 Ungraded Plugin • 15 min
- Try Out Programmatic Prompting
 Reading • 10 min
- Programmatic Prompting for Agents II
 Ungraded Plugin • 15 min
- Programmatic Prompting for Agents III
 Ungraded Plugin • 15 min
- Programmatic Prompting: Building Blocks for AI Agents
 Dialogue • 15 min
- Try Out the Customer Service Agent
 Reading • 10 min
- Giving Agents Memory
 Ungraded Plugin • 15 min
- Practicing Programmatic Prompting for Agents
 Ungraded Plugin • 15 min
- Practicing Programmatic Prompting for Agents (Solution)
 Ungraded Plugin • 15 min
- Adding Structure to AI Agent Outputs
 Video • 14 min
- Building Your First Agent
 Ungraded Plugin • 15 min
- AI Agent / Environment Interface
 Ungraded Plugin • 15 min
- AI Agent Feedback and Memory
 Ungraded Plugin • 15 min
- The Agent Loop: From Understanding to Thinking Like an Agent
 Dialogue • 15 min
- Learning More & Staying Connected
 Reading • 10 min
- Understanding Agentic AI Concepts
 Graded Assignment • 30 min

Module 2
AI Agents, Tools, Actions, & Language

Module 3
GAME: A Conceptual Framework for AI Agents

Module 4
Agent Tool Mangement

Module 5
Rethinking How Software is Built in the Age of AI Agents

Running the Code Samples in the Course

5:46 / 5:52 2x

Throughout the course, we will be using LiteLLM, a library that provides a unified interface for interacting with over 100 different LLMs including GPT-4, Claude, Gemini, and many others. By using a single interface, we can easily switch between different models without changing our code, making it perfect for testing different LLMs or having fallback options in production. This will also allow us to make our agent reusable across LLMs — although we may need to do some prompt engineering to make it work with a new LLM.

WE STRONGLY RECOMMEND USING GOOGLE COLAB WITH AN OPENAI API KEY FOR RUNNING THE CODE IN THIS COURSE.

API Key

You will need API keys for the LLM providers you plan to use. All of the examples in this course were written and tested for OpenAI's GPT-4o or GPT-4o-mini model. You are welcome to use another LLM, but you will need to figure out how to configure LiteLLM for that LLM. Usually, this is pretty easy and we provide a prompt that you can use in ChatGPT to get help.

You can read about how to get an OpenAI API key here: OpenAI API Keys. You can also use Anthropic's Claude, Google Gemini, Meta's LLama, or any other model that LiteLLM supports. However, we will only provide the configuration for using OpenAI in the tutorials.

Stop and Get an API Key

Once you have your API key, continue. Again, we recommend using OpenAI API Keys.

If you aren't using OpenAI…

If you decided NOT to use OpenAI, you will have to adapt the code samples in each lesson. Usually, this is as easy as changing this block of code at the top of each example:

For example, to change to Anthropic, you would change this:

```
# Important!!!
#
# <---- Set your 'OPENAI_API_KEY' as a secret over there with the "key" icon
#
#
import os
from google.colab import userdata
api_key = userdata.get('OPENAI_API_KEY')
os.environ['OPENAI_API_KEY'] = api_key
```

To this:

```
# Based on https://docs.litellm.ai/docs/providers/anthropic
#
# Important!!!
#
# <---- Set your 'ANTHROPIC_API_KEY' as a secret in Google Colab the "key" icon
#
#
import os
from google.colab import userdata
api_key = userdata.get('ANTHROPIC_API_KEY')
os.environ['ANTHROPIC_API_KEY'] = api_key
```

To change to Google Gemini, change your code to this:

```
# Important!!!
#
# <---- Set your 'GEMINI_API_KEY' as a secret in Google Colab the "key" icon
#
import os
from google.colab import userdata
api_key = userdata.get('GEMINI_API_KEY')
os.environ['GEMINI_API_KEY'] = api_key
```

Read the LiteLLM documentation if you need to use another provider: Lite LLM Provider Configuration

If you are not using OpenAI and are still running into issues…

You can use the following prompt in ChatGPT to help you adapt the code to work with another LLM:

```
Please ask me for my code and the LLM provider (e.g., Anthropic, OpenAI, etc.) that
I am using. Then, rewrite the code to show me how to configure it using an API key
for that provider. Only adapt the bare minimum of items in the configuration. Do not
change the code that interacts with the LLM if possible. I want to use LiteLLM
https://docs.litellm.ai/docs/providers.
```

Note: Many of the examples will only work with an LLM that supports function calling.

You will need to provide the API key as an environment variable to your code as described in the LiteLLM documentation using LiteLLM. I strongly recommend that you **do not hardcode your API keys** in your code.

Try Out Your API Key

Once you have an API key, you can use this Jupyter notebook on Google Colab to try out your key:
Example Code in Notebook

Most lessons will have a notebook AT THE END OF THE LESSON / MODULE that you can use to test the code.

How to Run the Code in the Tutorials

Using Google Colab (Recommended Method)

Google Colab provides the easiest way to run this agent code without setting up a local environment. Each lesson will have one or more Google Colab notebook links you can use to get started quickly.

Using Jupyter Notebooks Locally

If you prefer to use Jupyter notebooks on your local machine:

1. **Install Python**: Ensure you have Python 3.8+ installed on your system
2. **Create a virtual environment** (recommended):
3. **Install dependencies**:
4. **Set up API keys as environment variables** before starting Jupyter:
5. **Start Jupyter**:
6. **Create a new notebook** and paste the complete code in, preferably starting with the version stored in Google Colab.

Running as a Python Script

To run the agent code as a standalone Python script:

1. **Create a new Python file** (e.g., agent.py) with the agent code. You can copy / paste the code from the Google Colab notebook into a file in the order that it appears. You want JUST the code.
2. **Set environment variables** before running the script:
3. **Run the script**:
4. **Alternative: Use python-dotenv for easier API key management**:

Troubleshooting Common Issues

- **API Key Errors**: Ensure your API key is correctly set as an environment variable
- **Module Not Found**: Verify that you've installed all required packages
- **Rate Limiting**: If you encounter rate limits, add a short delay between API calls.
 - If you still receive Rate Limit Errors (or a 429 error) after adding this delay and you are using the OpenAI API, you may have exceeded your OpenAI API account's current API credit limits. To check your account's OpenAI API credit limits and increase them if necessary, please see these details. You can monitor your OpenAI API Usage and API spend in your OpenAI API account's "Usage" tab for investigation. Please note that OpenAI API billing is managed separately from OpenAI ChatGPT subscriptions.
 - **We strongly recommend using OpenAI APIs for this course**, as all code examples in this course are provided for this model. However, if you are blocked or unable to proceed, here are some details on Google Gemini APIs (and their free tier option). If you take this route, you'll need to translate all existing code examples in this course to match the Gemini API and Lite LLM Provider Configurations. Here is one starter code sample for how you could use Gemini APIs in a Google Colab Notebook.

```
# Install the necessary libraries using Colab's shell command system (!!).
# 'litellm' is a library that provides a unified interface for various LLM APIs.
!!pip install litellm

# 'google-genai' is the official, updated SDK for interacting with Google's Gemini models.
# The '-q' flag runs it quietly (hiding installation logs), and '-U' ensures it's upgraded to the latest version.
!!pip install -q -U google-genai

# Important!!!
#
# <---- Set your 'GEMINI_API_KEY' as a secret in Google Colab the "key" icon
#
import os
from google.colab import userdata
api_key = userdata.get('GEMINI_API_KEY')
os.environ['GEMINI_API_KEY'] = api_key

from litellm import completion
from google import genai

# The client gets the API key from the environment variable `GEMINI_API_KEY`.
client = genai.Client()

# Define a helper function to handle the interaction with the Gemini model.
def generate_response(prompt: str) -> str:
    """Call LLM to get response"""
    # Request the model to generate content based on the provided prompt string.
    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt,
    )
    return response.text

# Create an interactive text input box in Colab to ask the user what they want to prompt the AI with.
what_to_help_with = input("What do you need help with?")

# Call the helper function with the user's input and store the AI's response.
response = generate_response(what_to_help_with)

# Print the final text response from the Gemini model to the console.
```

Remember that Google Colab or Jupyter notebooks offer the most straightforward setup, especially for beginners or quick testing. They provide an interactive environment where you can execute code cells individually and see the results immediately.

Go to next item Completed

Like Dislike Report an issue

Go to next item